

9.216

Exponente: -495
Mantisa: 1.1875

Co



```
# Para asegurar la convergencia por Gauss-Seidel la matriz debe tener autovalores po
# y debe ser definida positiva, es decir, es igual a su traspuesta conjugada

A = np.array([
    [206, 361, 86],
    [361, 3685, 507],
    [86, 507, 139]
])

def conv_gauss_seidel(A):
    # Diagonal dominante
    conv1 = True
    base = np.append(False, np.full(A.shape[1] - 1, True))
    for i in range(A.shape[0]):
        idx = np.roll(base, i)
        if np.sum(np.abs(A[i, idx])) >= np.abs(A[i, i]):
            conv1 = False
            break

    if conv1:
        return True

    # Simétrica y definida positiva
```

```

if np.allclose(A, A.T) and np.all(np.linalg.eig(A)[0] > 0):
    return True

# Método con B
B = np.linalg.inv(np.tril(A)) # B = K^-1, donde K es la parte inferior de A más
M = B.dot(A)
eig = np.linalg.eig(np.eye(*M.shape) - M)[0]

return np.all(np.abs(eig) < 1)

aut_val = np.linalg.eig(A)

print(f'Autovalores: {np.round(aut_val[0],3)}')

A_T = A.T

print(A_T)
print(conv_gauss_seidel(A))

```

```

Autovalores: [3794.108  178.918   56.974]
[[ 206  361   86]
 [ 361 3685  507]
 [  86  507 139]]
True

```

Ejercicio 4:

In [23]:

```

A = np.array([
    [4, 3, 7, 1, 5],
    [2, 4, 3, 2, 5],
    [4, 3, 1, 3, 3],
    [4, 10, 8, 9, 3],
    [3, 9, 10, 6, 10]
])

b = np.array([
    [1],
    [7],
    [1],
    [2],
    [7]
]).reshape(-1)

print((b/np.diagonal(A))[-1])

```

```
0.7
```

Ejercicio 5 - No esta bien

Considere la iteración punto fijo

$$y_{k+1} = \sin(y_k) + 0.5$$

Se sabe que converge a un punto fijo en el intervalo entre 0.2 y 3. Si se parte del punto medio de dicho intervalo, estime el número de iteraciones necesarias para alcanzar el punto fijo con un error menos a 0.0000715.

In [15]:

```

tol = 0.0000715
fun = lambda y: np.sin(y) + 0.5
maxiter = 1000
x = (3 + 0.2)/2
xk = [x]
niter = 0

```

```

for k in range(maxiter):
    xk = np.append(xk, fun(xk[-1]))
    niter += 1
    if abs(xk[-1] - xk[-2]) <= tol: break

print(niter)

```

4

In [16]: `fun(xk[-1]), xk[-1]`

Out[16]: (1.4973004540863242, 1.49730127417024)

Ejercicio 6:

Considere la ecuación diferencial ordinaria (problema de valor inicial)

$$y' = f(t, y), \quad t \in (a, b)$$

$$y(a) = y_0$$

Se resuelve numéricamente usando el método Heun. Cuando se divide el intervalo en 6.330 sub-intervalos iguales, se obtiene $y(b)$ igual a 0,018. Al duplicar el número de sub-intervalos, se obtiene $y(b)$ igual a 0,014. Estime el número de sub-intervalos necesarios para que el error en sea menor a 10^{-6} . La respuesta debe ser un número entero.

In [35]:

```

import numpy as np

n = 2

N1 = 6.330 # cantidad de intervalos -> 1/delta_t
y1 = 0.018

N2 = 2*N1
y2 = 0.014

error = 1e-6

c = (y1 - y2)/((1 - 1/2**n) * (1/N1)**n) #calculo primero la constante

delta_t = (error/c)**(1/n) #el error

n_iter = 1/delta_t #las iteraciones

print(np.round(n_iter,3))

```

462.278

Ejercicio 8:

Considere la ecuación

$$x^3 - a = 0$$

con $a = 2$. Usando el método de Newton-Raphson y comenzando de $x=2$, estime el número de iteraciones necesarias para alcanzar un error menor a 10^{-6} .

In [37]: `f = lambda x: x**3 - 2`

```

deriv = lambda x: 3*x**2
x_ini = 2
tol = 1e-6
maxiter = 1000

iter = 0

xk = [x_ini]

for i in range(maxiter):
    iter += 1
    xk = np.append(xk, xk[-1] - f(xk[-1]) / deriv(xk[-1]))
    if abs(xk[-1] - xk[-2]) <= tol: break

print(iter)

```

5

Ejercicio 9:

Considere la función

$$f(x) = \cos(x)$$

en el intervalo $[1;5]$.

In [53]:

```

f = lambda x: np.cos(x)

a = 1
c = 5

h = (c - a)/2
b = c - h

ya, yb, yc = f(a), f(b), f(c)
hmin = (4 * yb - 3 * ya - yc) / (4 * yb - 2 * ya - 2 * yc) * h

print(f'Mínimo: {np.round(a + hmin,3)}')

```

Mínimo: 3.092

Ejercicio 10:

In [56]:

```

def fun(x):
    return np.abs(np.sin(x))

a = 1
b = 4

r = (-1 + np.sqrt(5))/2

c = a + (1 - r) * (b - a)
d = a + r * (b - a)

# como me piden el limite izquierdo solo me importa el valor que va a tomar a
if fun(c) > fun(d): a = c

print("Limite izquierdo:", np.round(a,3))

```

Limite izquierdo: 2.146

In []:

